Ders 6 Çalışma Özeti

Ders 6 Çalışma Özeti

Genel Konular

- Faks sıkıştırma ve standartlar (ekrandan, 00:10:00)
  - İletişim hattı üzerinden görüntü gönderilirken sıkıştırma önemlidir. Faksta metin de resim olarak, yani görüntü olarak gönderilir.
  - ITU-T (International Telecommunication Union) önce T2 (Grup 1) ve T3 (Grup 2) standartlarını geliştirdi. Bunların yerini T4 (Grup 3) ve T6 (Grup 4) aldı.
  - Grup 3 (T4): PSTN (Public Switched Telephone Network, 9600 baud). Grup 4 (T6): ISDN (Integrated Services Digital Network, 64K baud).
  - İki yöntemde de sıkıştırma oranı 10:1 ya da daha yüksektir. Önceden bir sayfanın iletimi yaklaşık 1 dakika sürerken bu standartlarla birkaç saniyeye inmiştir.
- Pels ve çözünürlük (ekrandan, 00:14:00)
  - Faks makinesi dokümanı satır satır tarar (her satır bir scanline) ve satırları "pels" (picture elements) adı verilen küçük siyah-beyaz noktalara çevirir.
  - Yatay çözünürlük: her zaman 8,05 pels/mm (205 pels/inç; $1'' = 25{,}4$ mm). 8,5 inç genişliğinde taranan her satır 1738 pels olur. T4'te tarama için önerilen satır genişliği 8,2 inçtir.
  - Dikey çözünürlük (kalite): standart mod 3,85 scanline/mm, fine mod 7,7 scanline/mm, very fine mod 15,4 scanline/mm.
- Sıkıştırmasız gönderim süresi örneği (ekrandan, 00:17:20)
  - 10 inç (254 mm) yüksekliğinde bir sayfa sıkıştırmadan gönderilirse standart modda 978 scanline, 170 sn (2,82 dk); fine modda 1956 scanline, 339 sn (5,65 dk); very fine modda 3912 scanline, 678 sn (11,3 dk) sürer. Satır başına pels 1664'tür.
  - Bu süreler faks iletişiminde sıkıştırmanın ne kadar önemli olduğunu gösterir.
- Grup 3 ve Modified Huffman (MH) (ekrandan, 00:20:00)
  - Grup 3 kodu, birçok doküman üzerinde siyah ve beyaz pels'lerin run-length'leri analiz edilerek elde edilmiştir. Her run uzunluğu için Huffman ile değişken uzunluklu kod atanmıştır.
  - En sık run uzunlukları 2, 3 ve 4 siyah pels olarak bulunmuş ve bunlara kısa kodlar verilmiştir. 2 ile 7 arasındaki run'lara sahip beyaz pels'lere de kısa kodlar verilmiştir (slaytta "daha uzun kodlar" yazıyor; bkz. Detaylı Açıklamalar).
  - Grup 3, RLE ve Huffman kodlamanın birleşimidir. Run değerleri uzun olabileceği için Huffman değiştirilmiştir; bu yüzden Grup 3'e MH (Modified Huffman) denir.
  - 0-63 run uzunlukları için Termination kodları, 64'ün katları için Make-up kodları kullanılır (tablolar ekrandan, 00:22:30).
- Run'ın kodlanması, EOL ve hata davranışı (ekrandan, 00:26:20, 00:28:30 ve 00:30:00)
  - Bir run kodu ya tek bir Termination kodundan ya da bir veya daha fazla Make-up kodu ve onu izleyen bir Termination kodundan oluşur.
  - Her tarama satırı ayrı kodlanır ve EOL koduyla biter. Her satırın soluna bir beyaz pel eklenir; bu, alıcının kodu çözerken belirsizliği önlemesi içindir.
  - Grup 3 hata düzeltmez ama hataların çoğunu bulur. Alıcı hatayı bulursa EOL'e kadar atlar; en fazla bir satır kaybedilir. EOL'ü göremezse yüksek hata oranı olduğunu kabul eder ve işlemi iptal eder. Kodlar 2-12 bit olduğundan 12 bit okuyup geçerli kod bulamayan alıcı hatayı anlar.
  - Her sayfanın sonunda 6 tane EOL vardır. Bu yönteme One-Dimensional Coding (tek boyutlu kodlama) denir.
  - Başarı görüntüye bağlıdır: metin ve siyah-beyaz görüntüde yüksek, kısa run'lardan

oluşan ve özellikle gray scale görüntüde düşüktür.
- İki boyutlu kodlama: MR (Modified READ) (ekrandan, 00:40:00 ve 00:44:50)
  - Gri tonlu görüntülerde tek boyutlu kodlama iyi sonuç vermediği için iki boyutlu kodlama geliştirilmiştir ve seçimliktir. READ: Relative Element Address Designate.
  - EOL'den sonraki bir bit sonraki satırın nasıl kodlanacağını söyler: 1 ise tek boyutlu, 0 ise iki boyutlu.
  - Kodlanan satıra coding line, bir üstündeki satıra reference line denir. Ardışık satırlar arasındaki fark az olduğu için üst satıra göre kodlamak daha iyi sıkıştırır.
  - 2D kodlama 1D'den daha az güvenilirdir; hata bütün dokümanı etkileyebilir. Bu yüzden T4'te (Grup 3) bir satır 1D kodlandıktan sonra en fazla $k-1$ satır 2D kodlanır (standart çözünürlükte $k=2$, yüksek çözünürlükte $k=4$). T6'da (Grup 4) böyle bir zorunluluk yoktur.
  - Kodlama satırındaki a_0, a_1, a_2 ve referans satırındaki b_1, b_2 konumlarına göre üç moddan biri seçilir: Pass, Vertical, Horizontal.
- Aritmetik kodlama (ekrandan, 01:20:00)
  - Sembollere tek tek kod verilmez; bütün input stream'e $[0, 1)$ aralığında tek bir sayı atanır. Kod "9746509" ise bu "0,9746509" demektir; "0," output stream'e yazılmaz.
  - İki geçişlidir: ilk geçişte sembol sıklıkları çıkarılır, ikinci geçişte sıkıştırma yapılır.
  - Her sembol, olasılığıyla orantılı bir alt aralık alır. Her yeni sembolle geçerli aralık daralır ve sonuçtaki sayı daha çok bitle ifade edilir.
- Aritmetik kodlamada EOF problemi ve tamsayı gerçekleştirmesi (ekrandan, 01:50:00, 01:51:20 ve 02:00:00)
  - Alt sınırı 0 olan bir sembol input'un sonundaysa kod çözücü bu sembolleri çözemez. Çözüm: olasılığı çok küçük ek bir EOF sembolü kullanmak ya da input'un boyutunu önce output'a yazmak.
  - Pratikte ondalık sayılar yerine 4 dijitlik tamsayı LOW ve HIGH değişkenleriyle çalışılır; en soldaki dijitler eşit olunca o dijit çıkışa yazılır ve sola öteleme yapılır.
- PPM (Prediction with Partial String Matching) (ekrandan, 02:10:00, 02:20:00 ve 02:30:00)
  - J. Cleary ve I. Witten tarafından ortaya atılmış, A. Moffat tarafından iyileştirilmiştir. "Order-N" Markov modelini kullanan içerik (context) tabanlı bir yöntemdir.
  - Model, sembole olasılık atamadan önce sembolün içeriğine (kendinden önce gelen N sembole) bakar. Kodlayıcı ve çözücü metnin ilerisini göremediği için geçmişle ilgilenir.
  - Statik içerik tabanlı yöntemlerde olasılıklar önceden hazırlanmış di-gram, tri-gram tablolarından gelir; adaptif yöntemlerde metin işlendikçe değişir.

Hocanın Özellikle Vurguladığı Kısımlar

- Faks konusundaki ayrıntılar ezberlenecek şeyler değildir
  - Hoca, bu standartların ayrıntısını sınavda "bunu tasarla" gibi sormayacağını; amacın veri transferi yapan her cihazda içeriğe göre farklı sıkıştırma algoritmaları geliştirildiğini göstermek ve bakış açısını genişletmek olduğunu söyledi.
- 2D kodlamanın üç modu ve geçiş kuralları
  - Mod belirlenirken önce Pass, sonra Vertical, sonra Horizontal kontrol edilir. Her kodlamadan sonra a_0, a_1, a_2, b_1, b_2 yer değiştirir.
  - Hoca slayttaki Vertical koşulunun "2'den fazla değil" yerine "3'ten fazla değil" olması gerektiğini söyledi (yedi durum: V0, VR1-3, VL1-3).
- 2D kodlama daha başarılıdır ama hataya açıktır
  - Hatayı bulamayan alıcı bütün sayfayı hatalı alabilir; bu yüzden Grup 3'te belli aralıklarla 1D satıra dönülür.
- Aritmetik kodlamanın özelliği
  - Hoca, tek bir sayıdan bütün metnin geri açılmasını çok şık bulduğunu ve aritmetik kodlamanın Huffman'a yakın, genelde ondan biraz daha iyi sonuç verdiğini söyledi.
  - Ortalama kod uzunluğu: Huffman'da $\sum P_i l_i$; aritmetik kodlamada output'un bit uzunluğu bölü input'taki sembol sayısı.
  - Kümülatif frekans önemli bir kavramdır; kod çözmede kullanılır.
- PPM'de order sayısı
  - Teoride N büyük (örn. 20.000) alınabilir; pratikte 1 ya da 2'dir, 3 bile nadirdir. Kombinasyon sayısı çok hızlı büyür ve ilk N karakter context'siz kodlanmak zorundadır.

Kısa Tekrar Notları

- ITU-T: T4 = Grup 3 (PSTN, 9600 baud), T6 = Grup 4 (ISDN, 64K baud); oran ≥ 10:1.
- Pels = picture elements. Yatay 8,05 pels/mm (205/inç), satırda 1738 pels. Dikey: 3,85 / 7,7 / 15,4 scanline/mm.
- 10 inçlik sayfa sıkıştırmasız: 2,82 / 5,65 / 11,3 dakika.
- MH = RLE + Huffman. Termination: 0-63; Make-up: 64'ün katları (2560'a kadar; 1792'den sonra siyah ve beyaz kodları aynı).
- 76 beyaz = 64 + 12 → `11011|001000`. 64 siyah = 64 + 0 → `0000001111|0000110111`.
- Her satır EOL ile biter, başına bir beyaz pel eklenir; sayfa sonunda 6 EOL.
- EOL'den sonraki bit: 1 → sonraki satır 1D, 0 → 2D.
- T4'te bir 1D satırdan sonra en fazla $k-1$ 2D satır ($k=2$ standart, $k=4$ yüksek çözünürlük); T6'da zorunluluk yok.
- Pass: b_2, a_1'in solunda → `0001` + b_1b_2; a_0 b_2'nin altına geçer.
- Vertical: $|a_1b_1| \leq 3$ → V0 `1`, VR1 `011`, VR2 `000011`, VR3 `0000011`, VL1 `010`, VL2 `000010`, VL3 `0000010`.
- Horizontal: $|a_1b_1| > 3$ → `001` + MH(a_0a_1) + MH(a_1a_2); a_0 a_2'nin yerine geçer.
- Aritmetik kodlama: NewHigh = OldLow + Range × HighRange(X); NewLow = OldLow + Range × LowRange(X).
- `a2a2a2a3` → $[0{,}8125;\ 0{,}825)$. `SWISS MISS` → 0,71753375.
- Kod çözme: sayının hangi aralıkta olduğuna bak, sembolü yaz, (kod − low)/range ile yeni kod; 0'a gelince dur.
- Alt sınırı 0 olan sembol sonda ise sorun çıkar → EOF sembolü ya da input boyutu.
- Tamsayı sürümü: ×10.000, HIGH'tan 1 çıkar; soldaki dijitler eşitse çıkışa yaz, LOW'a 0, HIGH'a 9 ötele. SWISS MISS → `717533750`.
- PPM: order-N context; büyükten küçüğe order'a "switch" edilir.

Detaylı Açıklamalar

- Pels ve pikselin farkı
  - Hoca, resimdeki her noktanın piksel olarak adlandırıldığını, faksta ise dokümanın satır satır (scanline) tarandığını ve bu siyah-beyaz noktalara pels dendiğini anlattı. Yatay ve dikey çözünürlük birbirinden farklıdır ve dikey çözünürlük seçilen moda göre değişir.
  - $8{,}05 \times 25{,}4 \approx 204{,}5$, yani yaklaşık 205 pels/inç; $8{,}05 \times 8{,}5 \times 25{,}4 \approx 1738$ pels (python ile doğrulandı).
- Gönderim süresi tablosu (ekrandan, 00:17:20)

Mod	Scanline	Pels/satır	Pels/sayfa	Süre (sn)	Süre (dk)
standart	978	1664	1.670M	170	2.82
fine	1956	1664	3.255M	339	5.65
very fine	3912	1664	6.510M	678	11.3

- Scanline sayısı $254 \times 3{,}85 \approx 978$, $254 \times 7{,}7 \approx 1956$, $254 \times 15{,}4 \approx 3912$'dir.
- **Ders sonrası not:** Hoca 1 inci bir yerde "2,54 mm" dedi; doğrusu 25,4 mm'dir (10 inç = 254 mm hesabı da buna göre yapıldı). Hoca standart mod için 977 scanline dedi; slaytta 978 yazar ($254 \times 3{,}85 = 977{,}9$).
- **Ders sonrası not:** Slayttaki standart mod pels/sayfa değeri (1.670M) diğer satırlarla tutarsızdır: $978 \times 1664 = 1.627.392$, yani büyük olasılıkla 1.627M olmalıdır. Süreler 9600 baud ile bölünerek bulunmuş gibidir ($1.627.392/9600 \approx 170$ sn; python ile doğrulandı).
- Termination ve Make-up tabloları (ekrandan, 00:22:30)
  - Tablolar yüzlerce farklı dokümanda siyah ve beyaz pels'lerin run'ları sayılarak hazırlanmıştır. Sık görülen run'lara kısa kod verilmiştir: beyazda 2, 3, 4 (`0111`, `1000`, `1011`), siyahta 2, 3 (`11`, `10`).
  - Run değeri 0 olabilir; örneğin 64 tane beyaz pel 64 + 0 olarak kodlanır.
  - Make-up tablosu 64'ten 2560'a kadar gider. 1792'den sonra siyah ve beyaz için aynı kod kullanılır. Hoca bunu, uzun run'ların genelde arka plan ya da bilgi taşımayan alan (örneğin fotokopinin siyah kenarları) olmasıyla açıkladı.
- Run kodlama örnekleri (ekrandan, 00:26:20)

```
12 beyaz            -> 001000
```

```
76 beyaz  = 64+12   -> 11011 | 001000
140 beyaz = 128+12  -> 10010 | 001000
64 siyah  = 64+0    -> 0000001111 | 0000110111
2561 siyah = 2560+1 -> 000000011111 | 010
```

- EOL ve satır başı beyaz pel örneği (ekrandan, 00:28:30)
  - 14 pels'lik satır: 3 siyah, 2 beyaz, 2 siyah, 7 beyaz. Başına bir beyaz pel eklenir: `1w 3b 2w 2b 7w EOL`.
  - Kod: `000111 | 10 | 0111 | 11 | 1111 | EOL` (1 beyaz, 3 siyah, 2 beyaz, 2 siyah, 7 beyaz). Bütün run'lar 64'ten küçük olduğu için yalnız Termination kodları kullanılır.
  - **Ders sonrası not:** Slayttaki EOL kodu `0000000001` olarak okunuyor. ITU-T T.4'te EOL büyük olasılıkla `000000000001` (11 sıfır ve bir 1, toplam 12 bit) olarak tanımlıdır.
- Beyaz run'lar hakkında slayt cümlesi
  - **Ders sonrası not:** Slayt "2 ve 7 arasındaki run değerlerine sahip beyaz pels'lere de daha uzun kodlar atanmıştır" diyor. Tabloya göre beyaz 2-7 run'ların kodları 4 bittir ve beyaz kodların en kısalarıdır; cümle büyük olasılıkla "kısa kodlar" anlamında okunmalıdır.
- Hata yakalama
  - Huffman kodun doğası gereği tek bir bit hatası alıcıyı senkronizasyon dışına çıkarabilir. Bozulan bit başka bir sembolün koduna denk gelirse hata fark edilmez; denk gelmezse fark edilir.
  - Hoca, bir karakterin üzerinden belki 30 scanline geçtiğini, tek bir satırlık hatanın gözle fark edilmeyeceğini; ama arka arkaya 10-15 satır kaybedilirse önemli bilginin kaybolacağını söyledi.
- 2D kodlamada modların ayrıntısı
  - a_0: kodlama satırındaki başlangıç (ilk başta satırın soluna eklenen beyaz pel), a_1: sonraki renk değişimi (ilk siyah), a_2: ondan sonraki renk değişimi. b_1, b_2: referans satırındaki birinci ve ikinci run'ların başlangıcı.
  - Pass mode (ekrandan, 00:44:50): b_2, a_1'in solundadır ve a_1 ile b_1 aynı hizada olamaz. Kod `0001` + b_1b_2 uzunluğu. Sonra a_0, b_2'nin altına taşınır; b_1, b_2 güncellenir; a_1, a_2 değişmez.
  - **Ders sonrası not:** Slayt ve hoca Pass mode'da `0001`'in arkasına b_1b_2 uzunluğunun yazıldığını söyledi. ITU-T T.4'te Pass mode kodu büyük olasılıkla yalnız `0001`'dir; b_1b_2 uzunluğu ayrıca gönderilmez, çünkü alıcı referans satırından b_2'yi kendisi bulabilir.
  - Vertical mode (ekrandan, 00:50:00): a_1 ile b_1 arası en fazla 3 pels. Kodlamadan sonra a_0 eski a_1'in yerine geçer.
  - Horizontal mode (ekrandan, 00:52:10): a_1 ile b_1 arası 3'ten fazla pels. Kod `001` + a_0a_1 (beyaz) ve a_1a_2 (siyah) run'larının MH kodu. Slayttaki örnekte 3 beyaz ve 7 siyah. Sonra a_0, a_2'nin yerine geçer, diğerleri sonraki run'lara güncellenir.
  - Ayrıca bir extension kodu vardır; hata anlaşılınca ya da gönderim iptal edilmek istenince kullanılır.
  - Hoca, bunun iki dizinin (siyah 0, beyaz 1) ve başlangıç konumlarının `if` kontrolleriyle karşılaştırılmasından ibaret olduğunu söyledi.
- 2D kodlama örneği (ekrandan, 00:58:10 ve 01:00:00)

```
1) |a1b1| = 1, a1 b1'in solunda   -> VL(1)      : 010
2) a1 ile b1 alt alta              -> V(0)       : 1
3) |a1b1| > 3, 3 beyaz, 4 siyah    -> Horizontal : 001 + 1000 + 011
4) b2, a1'in solunda               -> Pass       : 0001 + (2 beyaz run)
5) a1, b1'in 2 sağında             -> VR(2)      : 000011
6) a1, b1'in 2 solunda             -> VL(2)      : 000010
7) bir Horizontal daha, satır sonu -> EOL
```

- Satır bitince, $k-1$ tane 2D satır kodlandıysa sonraki satır 1D kodlanır; kodlanmadıysa bu satır referans, alttaki coding line olur. - Aritmetik kodlama örneği 1 (ekrandan, 01:30:00) - $a_1 \to [0;0{,}4)$, $a_2 \to [0{,}4;0{,}9)$, $a_3 \to [0{,}9;1)$. String `a2a2a2a3`. - a_2: Low $= 0+1\cdot 0{,}4 = 0{,}4$, High $= 0+1\cdot 0{,}9 = 0{,}9$. - a_2: Low $= 0{,}4+0{,}5\cdot 0{,}4 = 0{,}6$, High $= 0{,}4+0{,}5\cdot 0{,}9 = 0{,}85$. - a_2: Low $= 0{,}6+0{,}25\cdot 0{,}4 = 0{,}7$, High $= 0{,}6+0{,}25\cdot 0{,}9 = 0{,}825$. - a_3: Low $= 0{,}7+0{,}125\cdot 0{,}9 = 0{,}8125$, High $= 0{,}7+0{,}125\cdot 1 = 0{,}825$. Kod `8125` (python ile doğrulandı). - SWISS MISS örneği (ekrandan, 01:40:00 ve 01:43:30) - Sıklıklar: S 5, I 2, W 1, M 1, boşluk 1; toplam 10. Aralıklar: S $[0{,}5;1)$, W $[0{,}4;0{,}5)$, I $[0{,}2;0{,}4)$, M $[0{,}1;0{,}2)$, boşluk $[0;0{,}1)$. CumFreq: 5, 4, 2, 1, 0. - Kodlama

sonunda Low $= 0{,}71753375$, High $= 0{,}717535$. - Çözme: 0,71753375 → S; $(0{,}71753375 - 0{,}5)/0{,}5 = 0{,}4350675$ → W; $0{,}350675$ → I; $0{,}753375$ → S; $0{,}50675$ → S; $0{,}0135$ → boşluk; $0{,}135$ → M; $0{,}35$ → I; $0{,}75$ → S; $0{,}5$ → S; 0 → dur. Sonuç `SWISS MISS` (python ile doğrulandı). - EOF örneği (ekrandan, 01:50:00 ve 01:51:20) - a_1 0,001838 $[0{,}998162; 1)$, a_2 0,975 $[0{,}023162; 0{,}998162)$, a_3 0,023162 $[0; 0{,}023162)$. `a2a2a1a3a3` kodlanınca Low $= 0{,}99462270125$. - Çözerken a_2, a_2, a_1 bulunur; a_1'den sonra değer 0 olur ve çözme biter. a_3'ler kaybolur, çünkü a_3'ün alt sınırı 0'dır. - Tabloya olasılığı 0,000001 olan `eof` eklenince (`a3a3a3a3eof`) son sembol eof olur ve a_3'ler doğru çözülür. - PPM tablosu (ekrandan, 02:30:00) - `xyzzxyxyzzx` için order 4'ten order 0'a kadar context sayıları: order 0'da x 4, y 3, z 4; order 1'de x→y 3, y→z 2, y→x 1, z→z 2, z→x 2; order 4'te xyzz→x 2 vb. Bir z daha gelince z 5 olur ve yeni context'ler (yzzx→z, zzx→z, zx→z, x→z) eklenir (python ile doğrulandı). - Bir sembol için önce en büyük order'da context aranır; yoksa bir alt order'a "switch" edilir. Hiçbir order'da yoksa sembol yeni olarak eklenir. Switch'leri gösteren escape kodlarını hoca sonraki derse bıraktı.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.